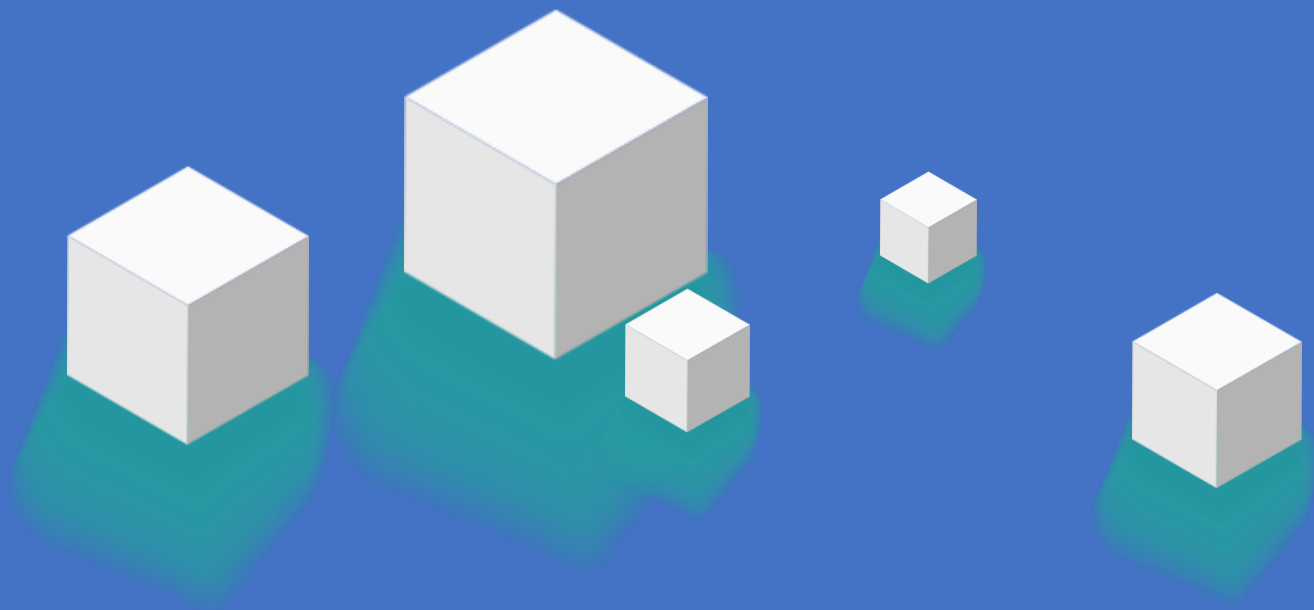


RAG相关面试辅导



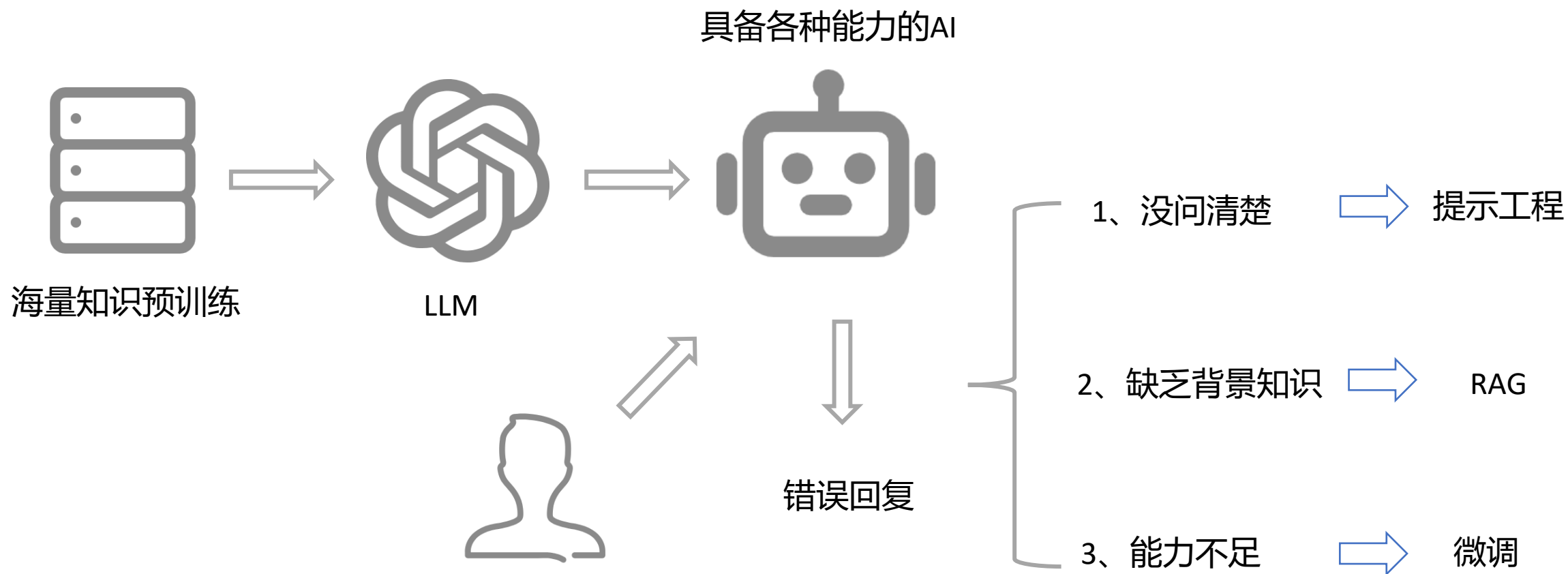
>> 今天的学习目标

RAG相关面试辅导

- 大模型应用开发三种模式
- 文档分块策略详解
- RAG系统架构图
- Embedding模型选择
- RAG调试与优化
- 多轮对话与Query改写
- 混合检索策略
- 重排序Rerank技术
- 知识库维护与迭代
- RAG系统评估方法
- 进阶技术GraphRAG
- 面试高频问题汇总

1. 大模型应用开发

Thinking: 提示工程 VS RAG VS 微调, 什么时候使用?



Q1：什么场景下应该选择 RAG 而不是 Fine-tuning?

Q1：什么场景下应该选择 RAG 而不是 Fine-tuning?

- 知识需要频繁更新: 如产品文档、FAQ，用 RAG 只需更新向量库
- 需要引用来源: 如客服系统需要告诉用户答案来自哪个文档
- 数据量有限: Fine-tuning 需要大量高质量数据，RAG 门槛更低
- 需要实时信息: 新闻、股票等实时数据无法通过训练固化到模型
- 预算有限: RAG 的实现成本远低于微调

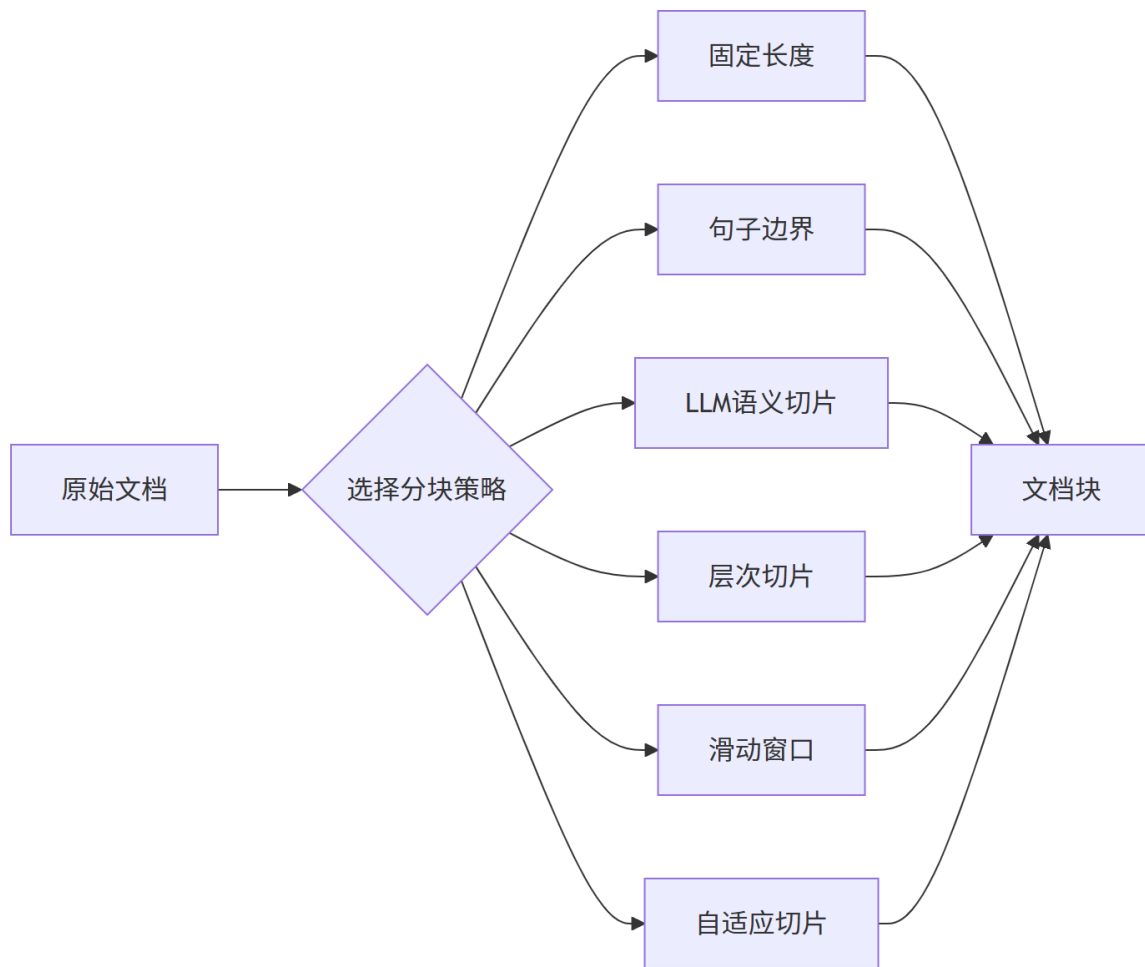
面试要点: 三种模式不是互斥的，实际项目中常常组合使用。

比如: RAG + Fine-tuning (微调模型，让它可以更好地利用检索结果)

或者 RAG + Prompt Engineering (优化检索后的提示词模板)。

2. 文档分块策略

文档分块 (Chunking) 是 RAG 系统的基础环节，分块质量直接影响检索效果。



2. 文档分块策略

文档分块 (Chunking) 是 RAG 系统的基础环节，分块质量直接影响检索效果。

策略	原理	优点	缺点	适用场景
固定长度	按字符数/Token数切分	实现简单，块大小可控	可能切断句子，破坏语义	结构简单的文本
句子边界	在句子结束符处切分	保持句子完整性	块大小不均匀	段落式文档
滑动窗口	固定窗口 + 重叠区域	保留上下文连续性	存储空间增加，有冗余	上下文依赖强的文本
层次切片	按文档结构(标题/章节)切分	保持文档逻辑结构	需要文档有清晰结构	技术文档、教材
LLM语义切片	用LLM判断语义边界	语义完整性最好	成本高，速度慢	高价值文档
自适应切片	根据内容密度动态调整	平衡各方面因素	实现复杂	混合类型文档

Q2：你项目中用了什么分块策略？为什么选它？

Q2：你项目中用了什么分块策略？为什么选它？

我在项目中使用了 滑动窗口 + 句子边界 的混合策略：

- 首先按句子边界切分，保证每个块语义完整
- 然后使用滑动窗口，设置20%重叠 (window=512, step=100)
- 重叠确保跨块的信息不会丢失

选择原因：

- 我们的知识库是产品FAQ，段落之间有上下文依赖
- 用户问题可能涉及多个连续段落的信息
- 20%重叠在存储开销和检索质量间取得平衡

分块大小经验值：

- 一般推荐 256-1024 tokens
- 小块: 检索精度高，但可能丢失上下文
- 大块: 上下文完整，但噪声多，检索精度下降
- 常见配置: chunk_size=512, overlap=50-100

Q3：请你描述下RAG系统的流程

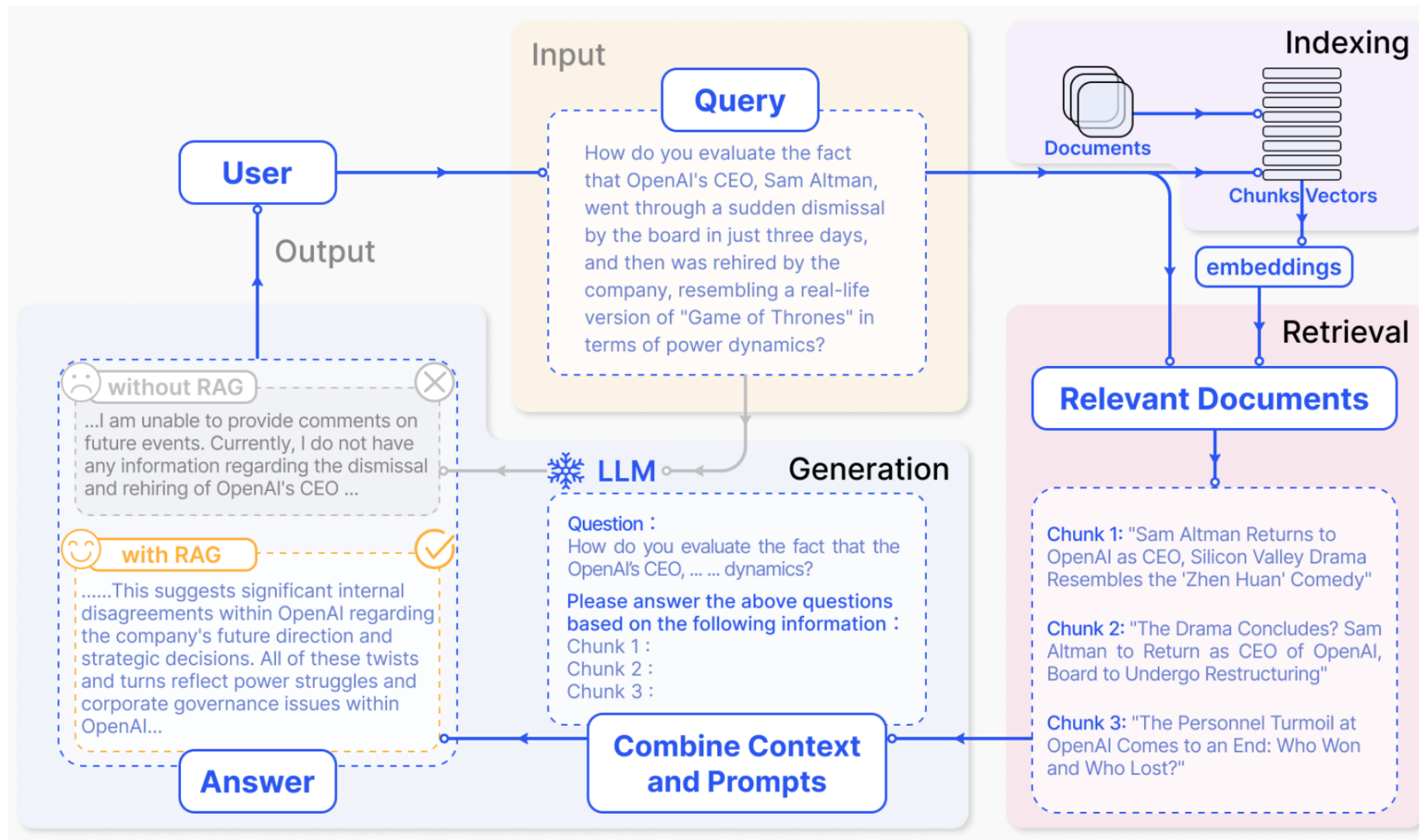
RAG的步骤：

Indexing => 如何更好地把知识存起来。

Retrieval => 如何在大量的知识中，找到一小部分有用的，给到模型参考。

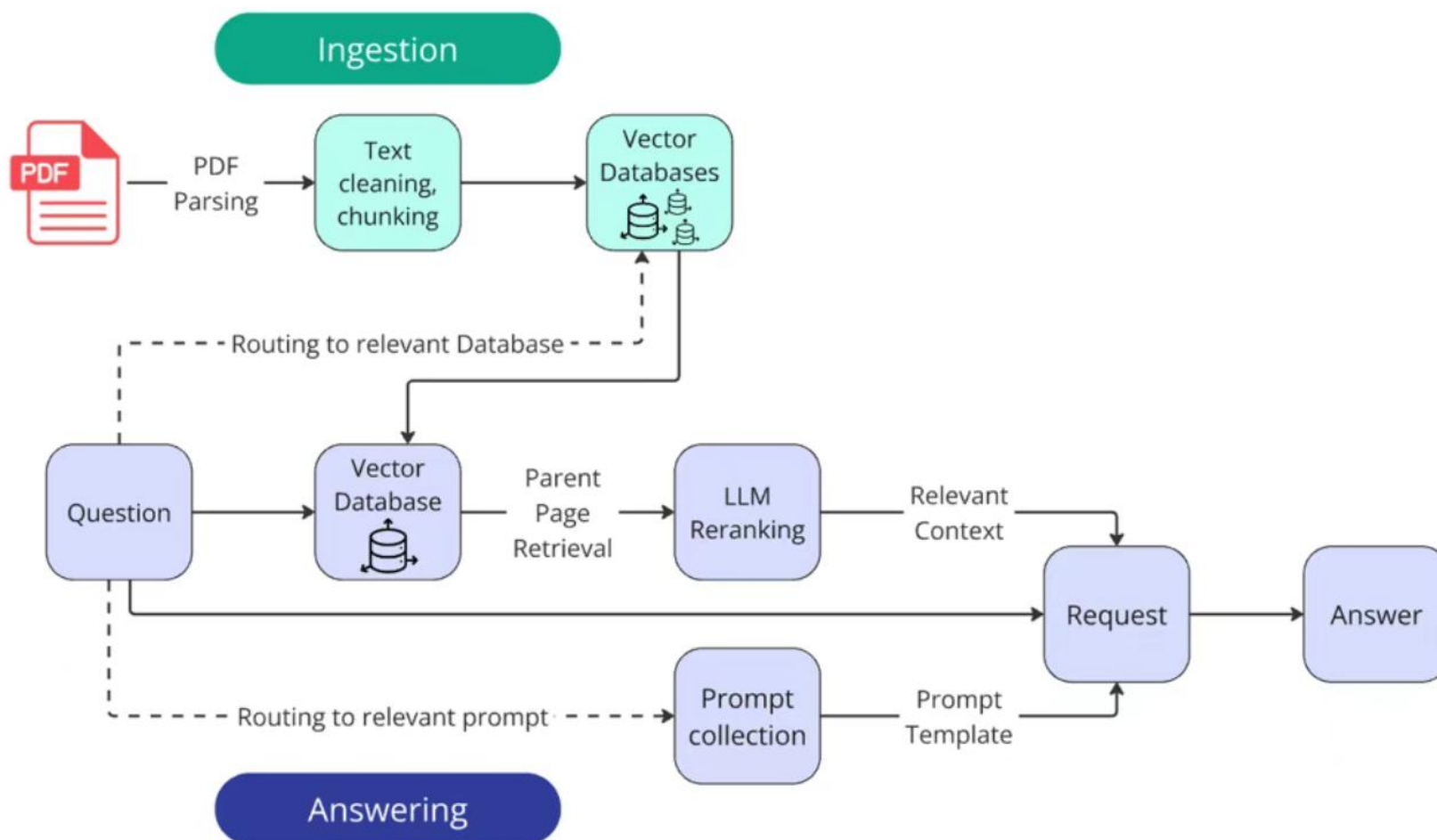
Generation => 如何结合用户的提问和检索到的知识，让模型生成有用的答案。

这三个步骤虽然看似简单，但在 RAG 应用从构建到落地实施的整个过程中，涉及较多复杂的工作内容



Q3：请你描述下RAG系统的流程

Thinking: 详细的步骤都有哪些？



Q3：请你描述下RAG系统的流程

关键步骤：

Step1，文档解析

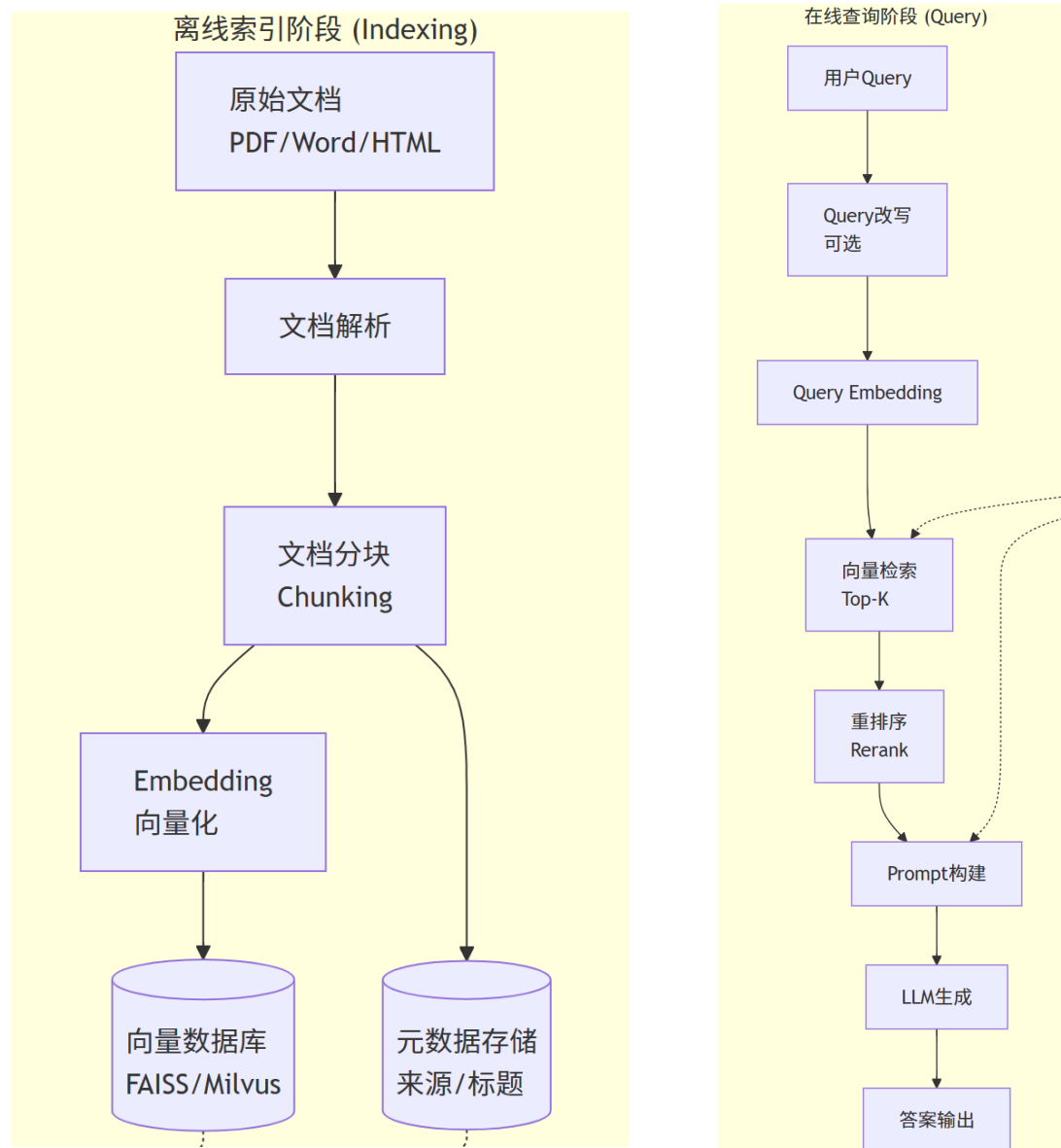
将PDF、Word、HTML等格式转换为纯文本。工具：PyPDF2, docx, BeautifulSoup。注意处理表格、图片等特殊内容。

Step2，文档分块 (Chunking)

将长文档切分为适合检索的小块。需要平衡块大小、上下文完整性、检索精度。

Step3，向量化 (Embedding)

使用Embedding模型将文本块转换为向量。常用模型：text-embedding-v4等。



Q3：请你描述下RAG系统的流程

Step4，向量存储

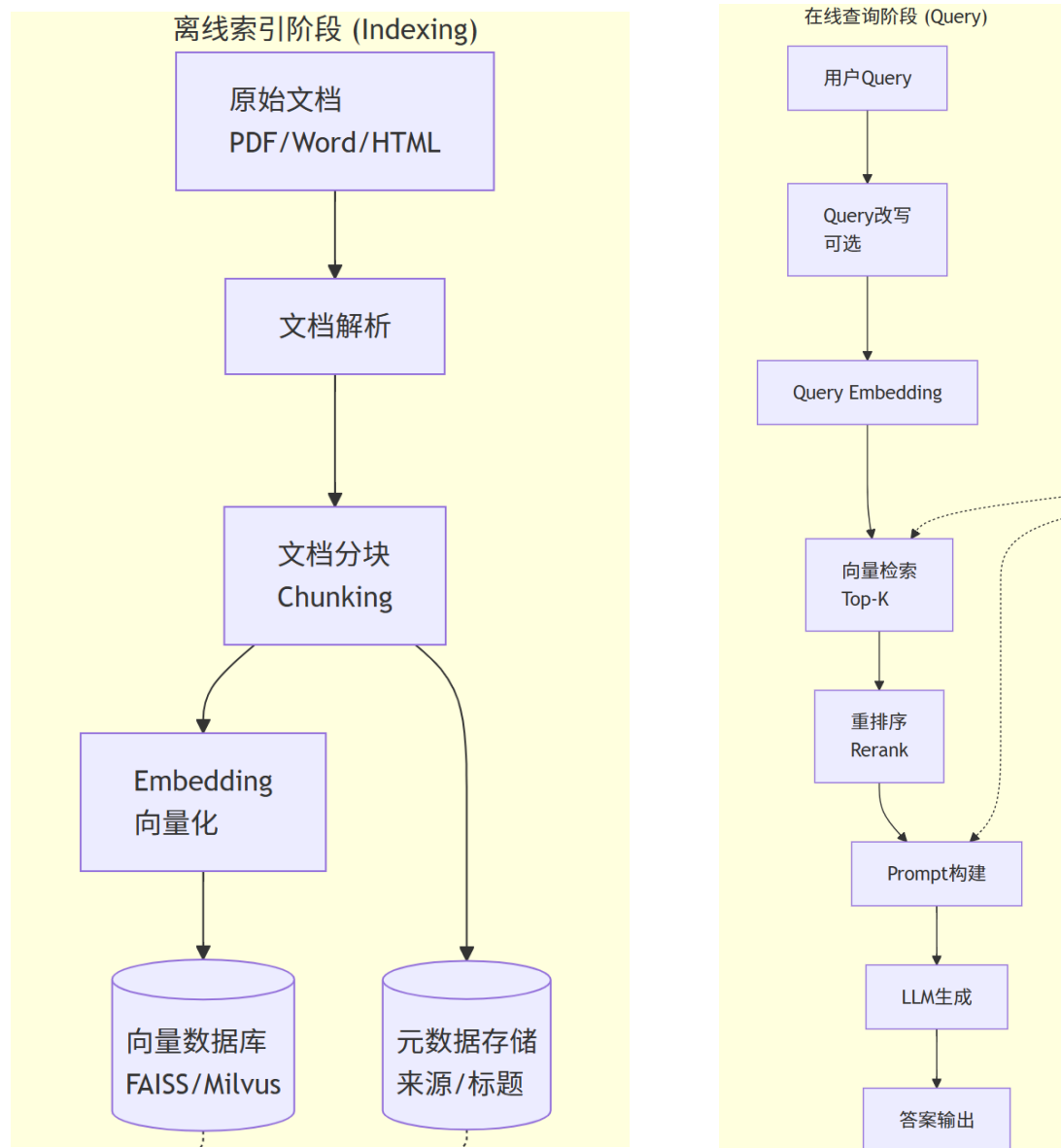
将向量存入向量数据库。FAISS (本地)、Milvus (分布式)、Pinecone (云服务)。同时存储元数据用于过滤和展示。

Step5，Query 改写 (可选)

处理模糊问题、补充上下文。使用LLM改写或扩展用户问题，提高检索召回率。

Step6，向量检索

计算Query向量与文档块向量的相似度，返回Top-K结果。
距离度量: 余弦相似度、L2距离、内积。



Q3：请你描述下RAG系统的流程

Step7，重排序 (Rerank)

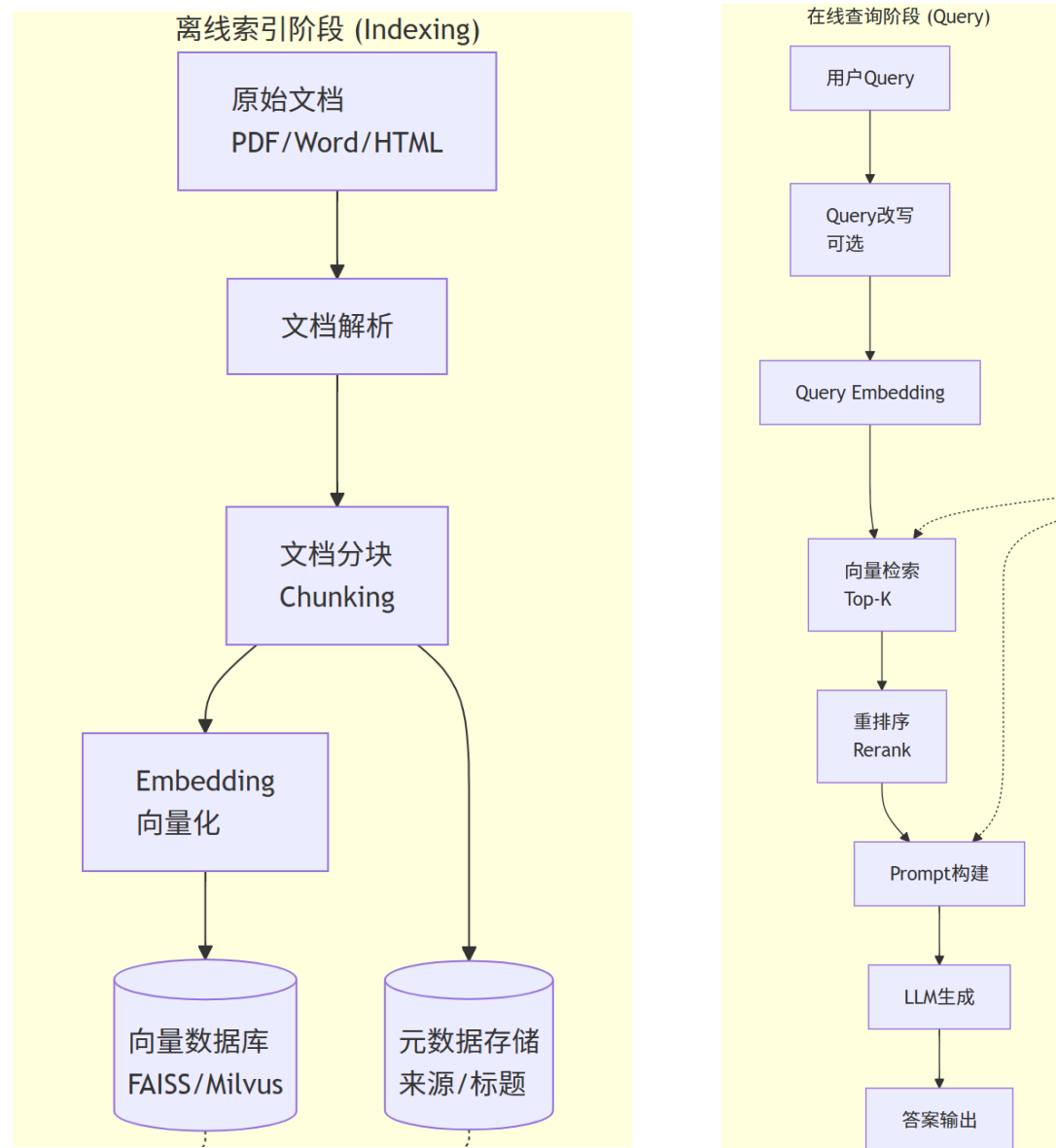
使用Cross-Encoder对Top-K结果精排，选出最相关的Top-N。显著提升最终效果。

Step8，Prompt 构建

将检索到的文档块拼接成Prompt中，作为LLM的上下文。注意控制总长度，避免超过模型上下文窗口。

Step9，LLM 生成

LLM基于Prompt生成最终答案。可以要求模型引用来源，提高可信度。



4. Embedding模型选择

模型	厂商	维度	特点	适用场景
text-embedding-ada-002	OpenAI	1536	通用性强，英文效果好	英文为主的场景
text-embedding-3-large	OpenAI	3072	最新模型，支持维度压缩	高精度需求
text-embedding-v4	阿里通义	512-1024	中文优化，性价比高	中文场景
BGE-large-zh	智源	1024	开源，中文效果优秀	私有化部署
M3E	Moka	768	开源，轻量级	资源有限场景
bge-m3	智源	1024	多语言，支持多种检索方式	多语言混合

主流 Embedding 模型对比

4. Embedding模型选择

Thinking: Embedding模型选择都有哪些考虑因素？

语言支持

- 中文场景: BGE、text-embedding-v4
- 英文场景: OpenAI系列
- 多语言: bge-m3

部署方式

- API调用: OpenAI、通义
- 私有化部署: BGE、M3E
- 混合: 都支持

性能指标

- 延迟: 本地部署 < API调用
- 吞吐: 取决于硬件/并发
- 精度: 需要在自己数据上测试

成本考量

- API按量付费，初期低
- 私有部署需GPU，长期划算
- 维度影响存储成本

Q4：你在项目中用了哪个 Embedding 模型？

Q4：你在项目中用了哪个 Embedding 模型？为什么选它？

我使用了阿里的 text-embedding-v4:

- 中文优化: 我们的知识库主要是中文文档，该模型在中文语义理解上表现优秀
- 维度可配: 支持512/1024维度，我选择1024维，在精度和存储间平衡
- 成本合理: API价格比OpenAI便宜，适合我们的预算
- 生态兼容: 与通义千问系列模型配合使用，接口统一

Q4：你在项目中用了哪个 Embedding 模型？

```
completion = client.embeddings.create(  
    model="text-embedding-v4",  
    input="查询文本",  
    dimensions=1024,  
    encoding_format="float"  
)
```

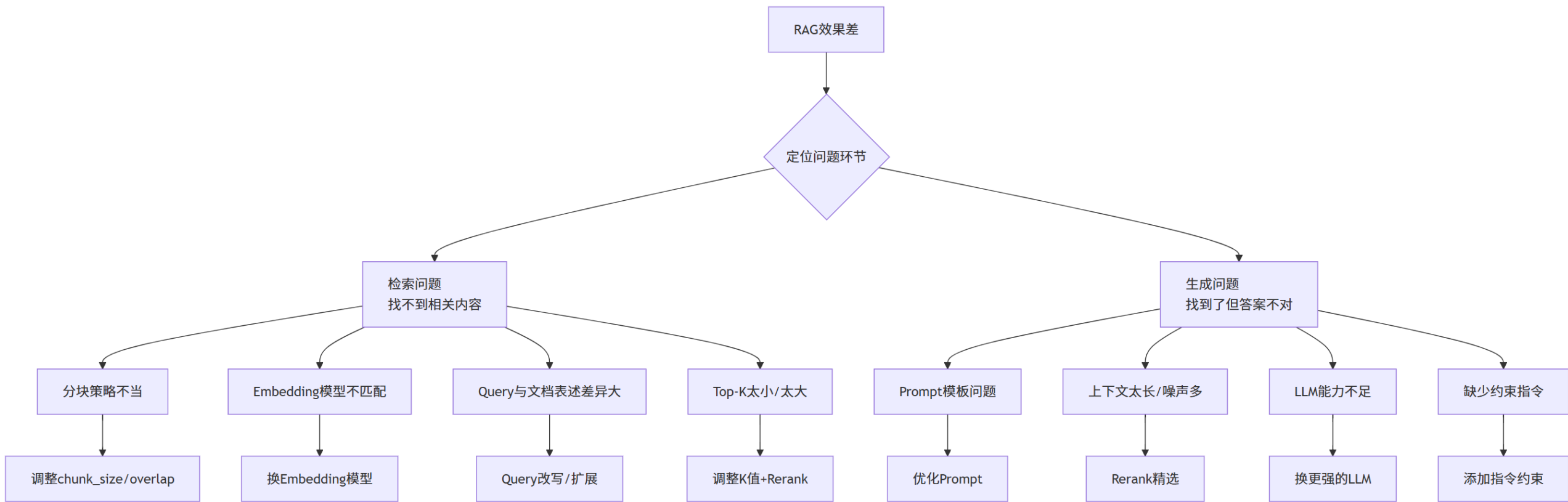
注意事项:

- Query和Document必须使用相同的Embedding模型
- 更换模型需要重建整个向量索引
- 建议在自己的数据集上做A/B测试选模型

Q5：如果RAG效果很差，你会从哪几个方面去调试？

Q5：如果你的 RAG 效果很差，你会从哪几个方面去调试？

会按照 RAG 的流程，逐步排查问题：



Q5：如果RAG效果很差，你会从哪几个方面去调试？

Step1 检索阶段调试

问题现象	可能原因	解决方案
召回内容不相关	Embedding模型对领域词理解差	换用领域微调的Embedding，或加同义词扩展
答案散落在多个块	分块太小，信息被切断	增大chunk_size，增加overlap
噪声太多	分块太大，混入无关内容	减小chunk_size，添加Rerank
用户口语化问题检索差	Query与文档表述风格差异	Query改写，HyDE假设文档生成

Q5：如果RAG效果很差，你会从哪几个方面去调试？

Step2 生成阶段调试

问题现象	可能原因	解决方案
答案与检索内容不符	LLM幻觉，未遵循上下文	强化Prompt指令："仅基于背景知识回答"
答案过于简短	Prompt未要求详细解释	添加输出格式要求
答案冗长有很多废话	上下文噪声多	Rerank精选，减少喂给LLM的内容
无法处理复杂推理	LLM能力不足	换用更强的模型，或添加CoT

Q5：如果RAG效果很差，你会从哪几个方面去调试？

Step3 调试工具与方法

调试检索效果的方法

```
def debug_retrieval(query, index, metadata, k=10):  
    """打印检索详情，帮助调试"""  
    query_vec = get_embedding(query)  
    distances, indices = index.search(  
        np.array([query_vec]).astype('float32'), k  
    )  
    print(f"Query: {query}")  
    print("-" * 80)  
    for rank, (idx, dist) in enumerate(zip(indices[0],  
        distances[0])):  
        if idx == -1:  
            continue
```

```
        doc = metadata[idx]  
        similarity = 1 / (1 + dist) # L2距离转相似度  
        print(f"Rank {rank+1} | 相似度: {similarity:.4f} | 距离: {dist:.4f}")  
        print(f"来源: {doc.get('source', 'N/A')}")  
        print(f"内容: {doc['text'][:100]}...")  
        print("-" * 40)  
    return indices, distances
```

- 先用debug_retrieval检查检索结果是否正确
- 如果检索结果好但生成差，优化Prompt
- 如果检索结果差，从分块/Embedding/Query改写入手
- 记录Bad Case，建立评估数据集持续改进

Q6：当用户的问题模糊， RAG 怎么优化？

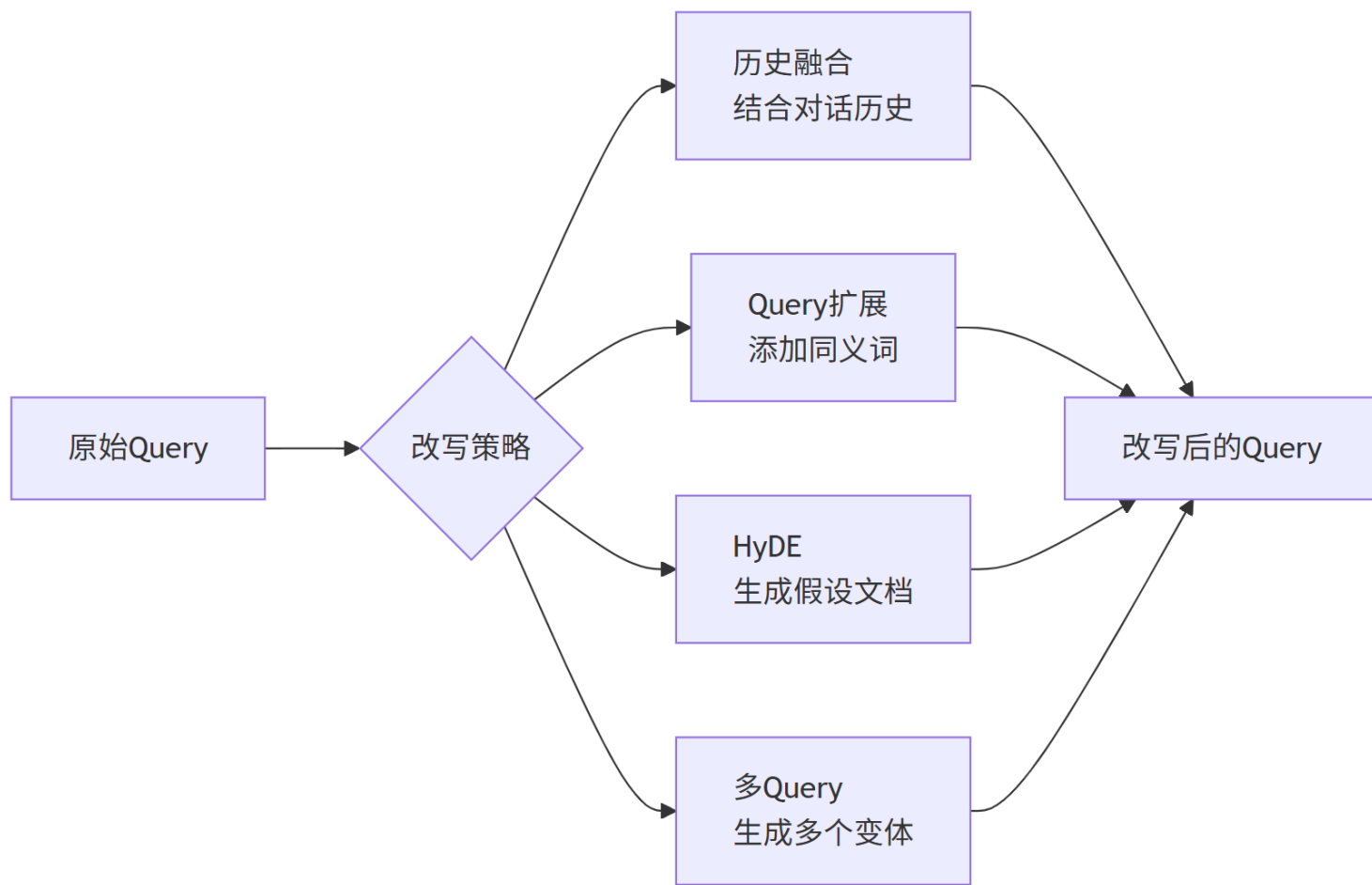
Q6：当用户的问题很模糊，或者依赖上一轮对话时， RAG 怎么优化？

Step1， 问题类型分析：

• 指代消解	• 省略补全	• 模糊问题
用户："它的退票政策是什么？ "	用户："那儿童票呢？ "	用户："怎么买票？ "
"它"指代上文提到的"迪士尼"	省略了"退票政策"的上下文	缺少具体场景 (线上/线下/团购)
解决：结合历史对话改写Query	解决：从历史中补充完整语义	解决：Query扩展或追问

Q6: 当用户的问题模糊, RAG 怎么优化?

Step2, Query 改写技术



Q6: 当用户的问题模糊, RAG 怎么优化?

历史融合改写示例

```
def rewrite_query_with_history(current_query,
chat_history, client):
    """结合对话历史改写Query"""
    # 构建改写Prompt
    history_str = "\n".join([
        f"用户: {h['user']}\n助手: {h['assistant']}"
        for h in chat_history[-3:] # 最近3轮
    ])
    prompt = f"""基于以下对话历史, 改写用户的最新
问题, 使其成为一个独立、完整、明确的问题。
```

[对话历史]

{history_str}

[最新问题]

{current_query}

[改写要求]

1. 补充省略的主语/宾语
2. 解析指代词 (它/这个/那个)
3. 补充上下文信息
4. 保持用户原始意图

改写后的问题: """

```
response = client.chat.completions.create(
    model="qwen-plus",
    messages=[{"role": "user", "content": prompt}],
    temperature=0
)
return response.choices[0].message.content
```

Q6：当用户的问题模糊， RAG 怎么优化？

历史融合改写示例

```
# 使用示例
history = [
    {"user": "迪士尼的门票多少钱？ ", "assistant": "迪士尼门票价格..."},
    {"user": "能退票吗？ ", "assistant": "关于退票政策..."}
]
current = "儿童票呢？ "
# 改写结果: "迪士尼儿童票的退票政策是什么？ "
```


Q6: 当用户的问题模糊, RAG 怎么优化?

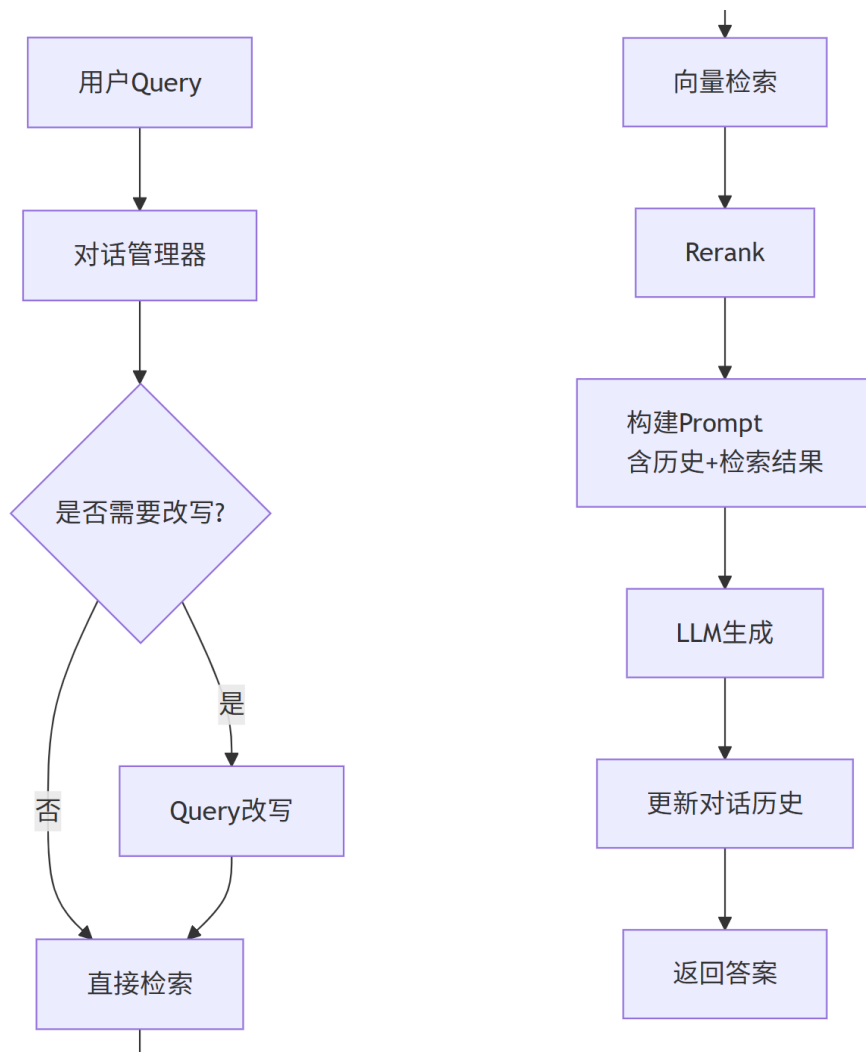
HyDE (假设文档嵌入)

```
def hyde_query_expansion(query, client):  
    """HyDE: 让LLM生成假设的答案文档, 用文档向量  
    检索"""  
  
    prompt = f"""请为以下问题生成一段可能的答案文  
    档(约100字):  
  
    问题: {query}  
  
    假设答案文档:"""
```

```
response = client.chat.completions.create(  
    model="qwen-plus",  
    messages=[{"role": "user", "content": prompt}],  
    temperature=0.7  
)  
  
hypothetical_doc =  
response.choices[0].message.content  
  
# 用假设文档的向量去检索, 而不是原始Query  
# 因为文档向量与文档向量更相似  
return get_embedding(hypothetical_doc)
```

Q6: 当用户的问题模糊, RAG 怎么优化?

Step3, 多轮对话 RAG 架构



实践建议:

- 对话历史不宜过长, 一般保留最近3-5轮
- 可以用LLM判断是否需要改写, 避免每次都改写
- 改写模型可以用较小的模型, 降低延迟
- 记录改写前后的Query, 便于调试

Q7： 向量检索有什么缺点？ 什么是混合检索？

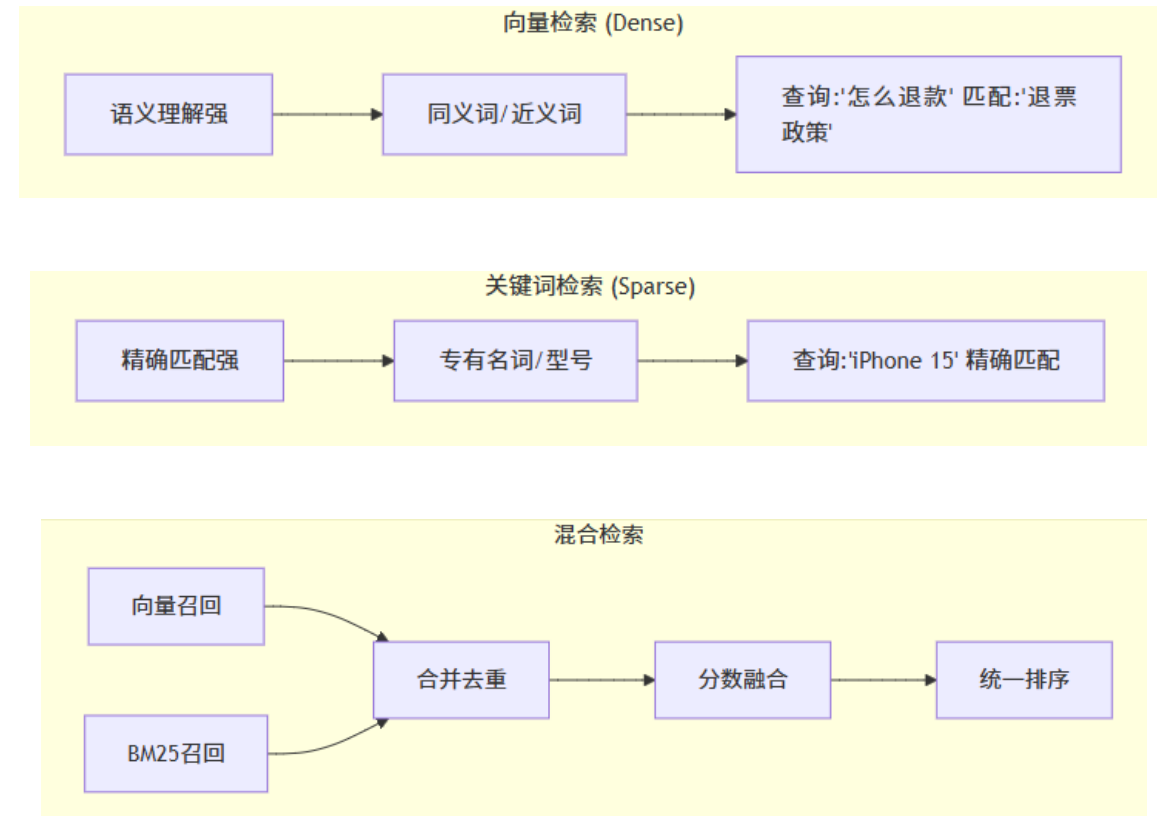
Q7： 你只用了向量检索吗？ 它有什么缺点？ 什么是混合检索？

向量检索的缺点：

- 对精确关键词匹配不敏感 (如产品型号、人名)
- 可能漏掉字面完全匹配的内容
- Embedding模型对领域专有词理解可能不准

混合检索: 结合向量检索和关键词检索 (BM25)，取长补短

Q7：向量检索有什么缺点？什么是混合检索？



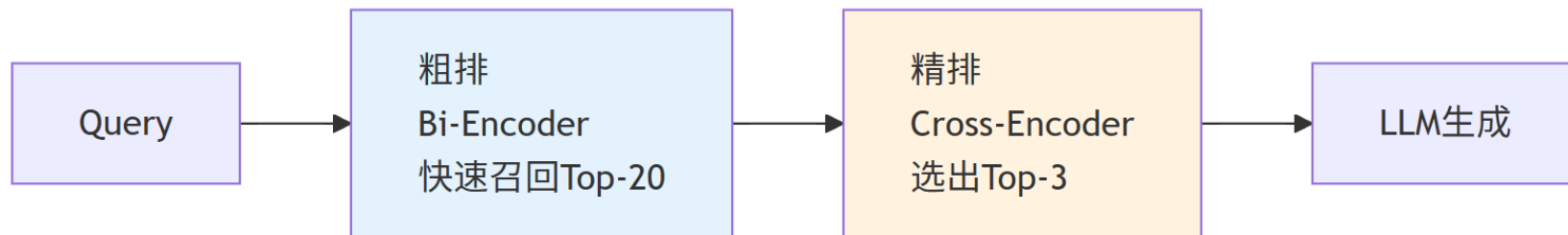
特性	向量检索	关键词检索 (BM25)	混合检索
语义理解	强	弱	强
精确匹配	弱	强	强
专有词处理	一般	好	好
实现复杂度	中	低	高
延迟	低	很低	中

Q8: 召回了 20 条文档，怎么确保给 LLM 的是最好的 3 条？

Q8: 检索召回了 20 条文档，你怎么确保喂给 LLM 的是最好的 3 条？

使用 Rerank (重排序) 技术，对初步召回的结果进行精排。

Thinking: 为什么需要 Rerank?



Q8: 召回了 20 条文档，怎么确保给 LLM 的是最好的 3 条？

Bi-Encoder (向量检索)

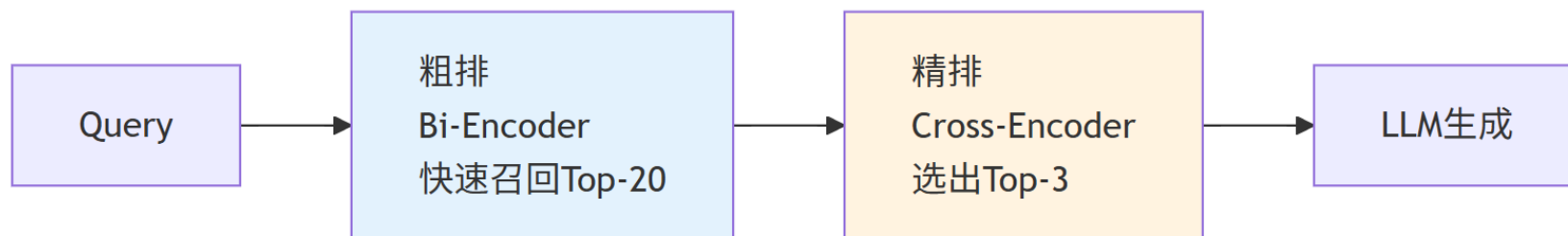
Query和Document分别编码，计算向量相似度

- 速度快，适合大规模召回
- Query和Doc独立编码，交互信息少
- 精度相对较低

Cross-Encoder (Rerank)

Query和Document拼接后一起编码，直接输出相关性分数

- 精度高，能捕捉细粒度交互
- 速度慢，只能处理少量候选
- 适合对Top-K精排



Q8: 召回了 20 条文档，怎么确保给 LLM 的是最好的 3 条？

使用 BGE-Reranker

```
from transformers import AutoModelForSequenceClassification,
AutoTokenizer

import torch

class Reranker:
    def __init__(self, model_name="BAAI/bge-reranker-large"):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model =
AutoModelForSequenceClassification.from_pretrained(model_name)
        self.model.eval()

    def rerank(self, query, documents, top_n=3):
        """对文档列表重排序，返回Top-N"""
        # 构建 Query-Document 对
        pairs = [[query, doc['text']] for doc in documents]
```

```
# 编码
inputs = self.tokenizer(
    pairs,
    padding=True,
    truncation=True,
    max_length=512,
    return_tensors='pt'
)

# 推理
with torch.no_grad():
    scores = self.model(**inputs).logits.squeeze(-1).tolist()

# 按分数排序
scored_docs = list(zip(documents, scores))
scored_docs.sort(key=lambda x: x[1], reverse=True)
return scored_docs[:top_n]
```

Q8: 召回了 20 条文档，怎么确保给 LLM 的是最好的 3 条？

```
# 使用示例
reranker = Reranker()
top_docs = reranker.rerank(
    query="迪士尼退票政策",
    documents=retrieval_results, # 初步召回的20条
    top_n=3
)
```


Q8: 召回了 20 条文档，怎么确保给 LLM 的是最好的 3 条？

使用 API (Cohere Rerank)

```
import cohere

co = cohere.Client('your-api-key')

def rerank_with_cohere(query, documents, top_n=3):
    """使用Cohere API进行重排序"""
    docs_text = [doc['text'] for doc in documents]

    response = co.rerank(
        model='rerank-multilingual-v2.0',
        query=query,
        documents=docs_text,
        top_n=top_n
    )
```

```
results = []
for r in response.results:
    results.append({
        'document': documents[r.index],
        'relevance_score': r.relevance_score
    })

return results
```

Q8: 召回了 20 条文档，怎么确保给 LLM 的是最好的 3 条？

完整的召回-精排流程

```
def rag_with_rerank(query, index, metadata, reranker,
                    recall_k=20, rerank_n=3):
    """带重排序的RAG流程"""
    # Step 1: 向量检索 (粗排，召回Top-20)
    query_vec = get_embedding(query)
    distances, indices = index.search(
        np.array([query_vec]).astype('float32'), recall_k
    )
    candidates = []
    for idx in indices[0]:
        if idx != -1:
            candidates.append(metadata[idx])
    print(f"粗排召回 {len(candidates)} 条")
```

```
# Step 2: Rerank (精排，选出Top-3)
reranked = reranker.rerank(query, candidates, top_n=rerank_n)

print(f"精排选出 {len(reranked)} 条:")
for doc, score in reranked:
    print(f" - {score:.4f}: {doc['text'][:50]}...")

# Step 3: 构建Prompt，调用LLM
context = "\n\n".join([doc['text'] for doc, _ in reranked])
# ... 后续生成流程
```

Q8: 召回了 20 条文档，怎么确保给 LLM 的是最好的 3 条？

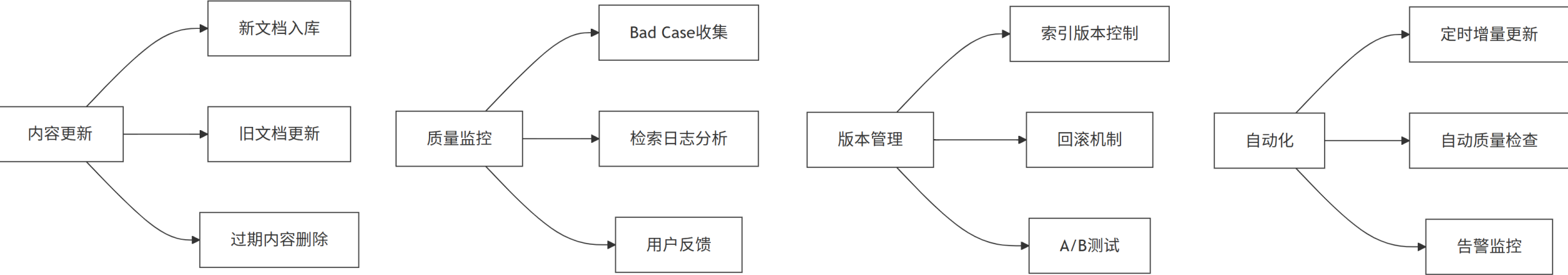
Rerank 实践建议:

- 召回数量 (recall_k) 一般设置为最终需要数量的5-10倍
- Rerank模型选择: 中文推荐 bge-reranker, 多语言用 Cohere
- Rerank会增加延迟，需要在效果和速度间权衡
- 可以设置分数阈值，过滤低相关性结果

Q9：系统上线后，你怎么维护和迭代你的知识库？

Q9：系统上线后，你怎么维护和迭代你的知识库？

知识库维护是一个持续的过程，包括以下几个方面：



Q9：系统上线后，你怎么维护和迭代你的知识库？

增量更新策略

```
import hashlib
from datetime import datetime

class KnowledgeBaseManager:
    def __init__(self, index, metadata_store):
        self.index = index
        self.metadata_store = metadata_store
        self.doc_hashes = {} # 文档hash，用于检测变更

    def compute_hash(self, text):
        """计算文档内容hash"""
        return hashlib.md5(text.encode()).hexdigest()
```

```
def add_document(self, doc_id, text, metadata):
    """添加新文档"""
    doc_hash = self.compute_hash(text)

    # 检查是否已存在
    if doc_id in self.doc_hashes:
        if self.doc_hashes[doc_id] == doc_hash:
            print(f"文档 {doc_id} 未变更，跳过")
            return
        else:
            print(f"文档 {doc_id} 已更新，先删除旧版本")
            self.delete_document(doc_id)

    # 生成向量
    embedding = get_embedding(text)
```

Q9: 系统上线后, 你怎么维护和迭代你的知识库?

```
# 添加到索引
vector_id = len(self.metadata_store)
self.index.add_with_ids(
    np.array([embedding]).astype('float32'),
    np.array([vector_id])
)

# 存储元数据
self.metadata_store.append({
    'doc_id': doc_id,
    'text': text,
    'metadata': metadata,
    'created_at': datetime.now().isoformat(),
    'hash': doc_hash
})
```

```
self.doc_hashes[doc_id] = doc_hash
print(f"文档 {doc_id} 已添加")

def delete_document(self, doc_id):
    """删除文档 (标记删除)"""
    for i, doc in enumerate(self.metadata_store):
        if doc.get('doc_id') == doc_id:
            doc['deleted'] = True
            doc['deleted_at'] = datetime.now().isoformat()
            break

    if doc_id in self.doc_hashes:
        del self.doc_hashes[doc_id]
```

Q9：系统上线后，你怎么维护和迭代你的知识库？

```
def rebuild_index(self):
    """重建索引 (清理已删除文档)"""
    active_docs = [
        doc for doc in self.metadata_store
        if not doc.get('deleted', False)
    ]

    # 重新构建索引
    # ... 实现省略

    print(f"索引重建完成，共 {len(active_docs)} 条有效文档")
```

Q9：系统上线后，你怎么维护和迭代你的知识库？

Q9 plus：维护知识库能否通过 Agent RL

Agent RL的核心思路：让Agent从环境反馈中学习改进策略。

关键是定义好：状态、动作、奖励。

方案A：基于反馈的Prompt优化

- 最轻量的"RL"方式，不涉及模型训练：
- 收集用户反馈（采纳/修改/拒绝）
- 分析Bad Case的模式
- 迭代优化System Prompt
- 加入成功案例作为Few-shot

方案B：RLHF微调

- 用人类反馈训练奖励模型，再优化策略：
- 收集成对比较数据（好回答vs差回答）
- 训练奖励模型(Reward Model)
- 用PPO等算法优化策略模型
- 需要大量数据和算力

Q9：系统上线后，你怎么维护和迭代你的知识库？

对于企业场景，推荐方案A + 选择性微调的组合：

轻量级Agent RL实现

class AgentFeedbackLoop:

def __init__(self):

self.feedback_db = FeedbackDatabase()

self.prompt_version = "v1.0"

def collect_feedback(self, task_id, agent_output, user_action, user_correction):

"""收集用户反馈"""

feedback = {

"task_id": task_id,

"agent_output": agent_output,

"user_action": user_action, # accept/modify/reject

"user_correction": user_correction,

"prompt_version": self.prompt_version

}

self.feedback_db.save(feedback)

def analyze_failures(self):

"""分析失败模式"""

failures = self.feedback_db.get_failures(limit=100)

用LLM分析失败模式

analysis = llm.analyze(

prompt="分析Agent输出被用户拒绝的原因，归纳共性问题",

data=failures

)

return analysis

def generate_improved_prompt(self, analysis):

"""基于分析生成改进的Prompt"""

improved = llm.generate(

prompt="""基于以下问题分析，改进System Prompt：

问题分析：{analysis}

当前Prompt：{current_prompt}

要求：针对性解决已发现的问题，保持其他功能不变"""

)

return improved

Q9：系统上线后，你怎么维护和迭代你的知识库？

Step1, 收集反馈 (collect_feedback)

场景： Agent 写了一句日报：“今日交易额 50000。” 业务人员（用户）看了一眼，觉得不行，修改为：“今日交易额 50,000.00 元。”

代码在做什么： 系统悄悄把这次交互记在小本本（数据库）上：

Agent写了啥： 50000

用户动作： Modify（修改）

用户改成啥： 50,000.00 元

当时的手册版本： v1.0

这就好比导师改了实习生的作业，系统把“错题本”和“正确答案”都存下来了。

Q9：系统上线后，你怎么维护和迭代你的知识库？

Step2, 分析失败模式 (analyze_failures)

场景：过了一周，数据库里积累了100条类似的修改记录。

代码调用一个更聪明的大模型（比如Qwen-Max或Claude 4.5 Sonnet）来当“教导主任”，分析这些错题。

代码在做什么 => 它把这100个错误扔给大模型问：“你看看，这个Agent老是被用户纠正，到底哪儿做的不对？”

大模型分析结果 => “共性问题发现：Agent输出金额时，普遍缺少千分位分隔符，且经常漏掉货币单位。”

这步不是简单的统计，而是让AI去归纳错误规律。不是针对某一次错误，而是发现一类错误。

Q9：系统上线后，你怎么维护和迭代你的知识库？

Step3，生成改进的Prompt

场景：既然发现了问题是“没写千分位”和“漏单位”，那就得改《操作手册》（System Prompt）。

代码在做什么 =》 代码自动把“旧手册”和“刚才的分析结果”放在一起，让大模型写一个新的手册。

旧 Prompt：你是一个金融助理，请根据数据生成日报。

新 Prompt：你是一个金融助理，请根据数据生成日报。

注意：涉及金额数字时，必须使用千分位分隔符（如 1,000），并严格标注货币单位（如 元/USD）。

通过以上方式形成闭环。下一次Agent再工作时，调用的是v1.1版的Prompt。

Summary

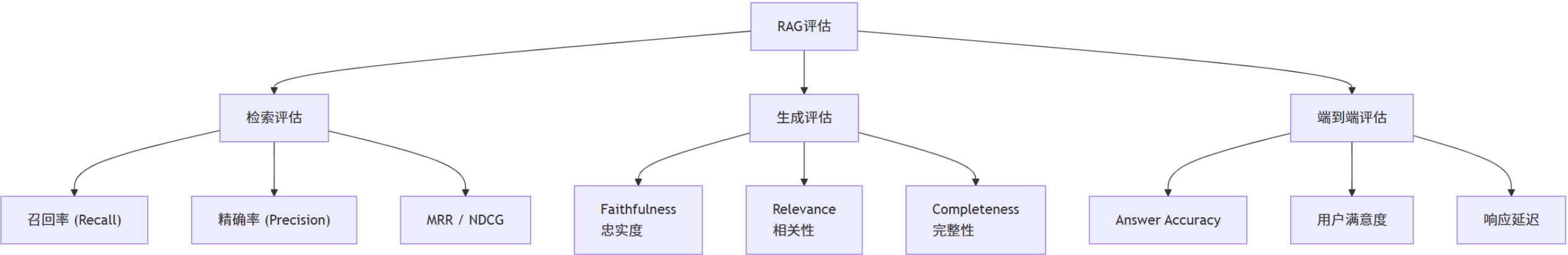
维护最佳实践:

- 定期审核: 每周/月审核Bad Case, 识别系统性问题
- 增量更新: 避免全量重建, 使用增量方式更新索引
- 版本控制: 保留历史版本索引, 支持快速回滚
- 文档生命周期: 设置过期时间, 自动标记/清理过期内容
- 监控告警: 检索空结果率、用户负反馈率等指标超阈值时告警

Q10：如何评估一个 RAG 系统的好坏？

Q10：如何评估一个 RAG 系统的好坏？

RAG系统的评估需要从检索质量和生成质量两个维度进行:



指标	定义	计算公式	适用场景
Recall@K	Top-K中包含相关文档的比例	相关文档数 / 总相关文档数	评估召回能力
Precision@K	Top-K中相关文档的比例	相关文档数 / K	评估精准度
MRR	第一个相关文档的排名倒数	1 / (第一个相关文档的排名)	单一正确答案场景
NDCG@K	考虑相关性等级的排序质量	DCG / IDCG	有相关性分级的场景

Q10: 如何评估一个 RAG 系统的好坏?

什么是 RAGAS?

RAGAS (Retrieval Augmented Generation Assessment) 是一个专门用于评估 RAG 系统的开源框架, 由 Exploding Gradients 团队开发。

核心特点:

- 无需人工标注: 使用 LLM 自动评估, 大幅降低评估成本
- 端到端评估: 同时评估检索质量和生成质量
- 指标全面: 提供 Faithfulness、Answer Relevancy、Context Precision 等核心指标
- 易于集成: 与 LangChain、LlamaIndex 等主流框架无缝对接

安装: `pip install ragas`

GitHub: <https://github.com/explodinggradients/ragas>

Q10: 如何评估一个 RAG 系统的好坏?

生成质量指标 (RAGAS框架)

- Faithfulness (忠实度)

答案是否基于检索到的内容，而非幻觉

评估方法: 用LLM判断答案中的每个声明是否能在上下文中找到支撑

- Answer Relevance (答案相关性)

答案是否回答了用户的问题

评估方法: 用LLM根据答案反向生成问题，与原问题比较相似度

Q10: 如何评估一个 RAG 系统的好坏?

- Context Relevance (上下文相关性)

检索到的内容是否与问题相关

评估方法: 计算上下文中与问题相关的句子比例

- Context Recall (上下文召回)

检索是否召回了回答问题所需的所有信息

评估方法: 对比标准答案, 检查所需信息是否被检索到

Q10: 如何评估一个 RAG 系统的好坏?

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_relevancy,
    context_relevancy,
    context_recall
)
from datasets import Dataset

def evaluate_rag_system(eval_data):
    """
    使用RAGAS评估RAG系统
    eval_data: 评估数据列表, 每条包含:
        - question: 用户问题
        - answer: RAG生成的答案
        - contexts: 检索到的上下文列表
        - ground_truth: 标准答案 (可选)
    """
```

```
# 转换为Dataset格式
dataset = Dataset.from_dict({
    'question': [d['question'] for d in eval_data],
    'answer': [d['answer'] for d in eval_data],
    'contexts': [d['contexts'] for d in eval_data],
    'ground_truth': [d.get('ground_truth', '') for d in eval_data]
})

# 评估
result = evaluate(
    dataset,
    metrics=[
        faithfulness,
        answer_relevancy,
        context_relevancy,
        context_recall
    ]
)

return result
```

Q10: 如何评估一个 RAG 系统的好坏?

```
# 评估数据示例
```

```
eval_data = [
```

```
{
```

```
    'question': '迪士尼门票可以退吗? ',
```

```
    'answer': '迪士尼门票原则上不退不换, 但在特殊情况下可申请...',
```

```
    'contexts': ['迪士尼门票一经售出, 原则上不予退换...'],
```

```
    'ground_truth': '门票原则上不退, 特殊情况可改期或退款'
```

```
},
```

```
# 更多评估样本...
```

```
]
```

```
results = evaluate_rag_system(eval_data)
```

```
print(results)
```

Summary

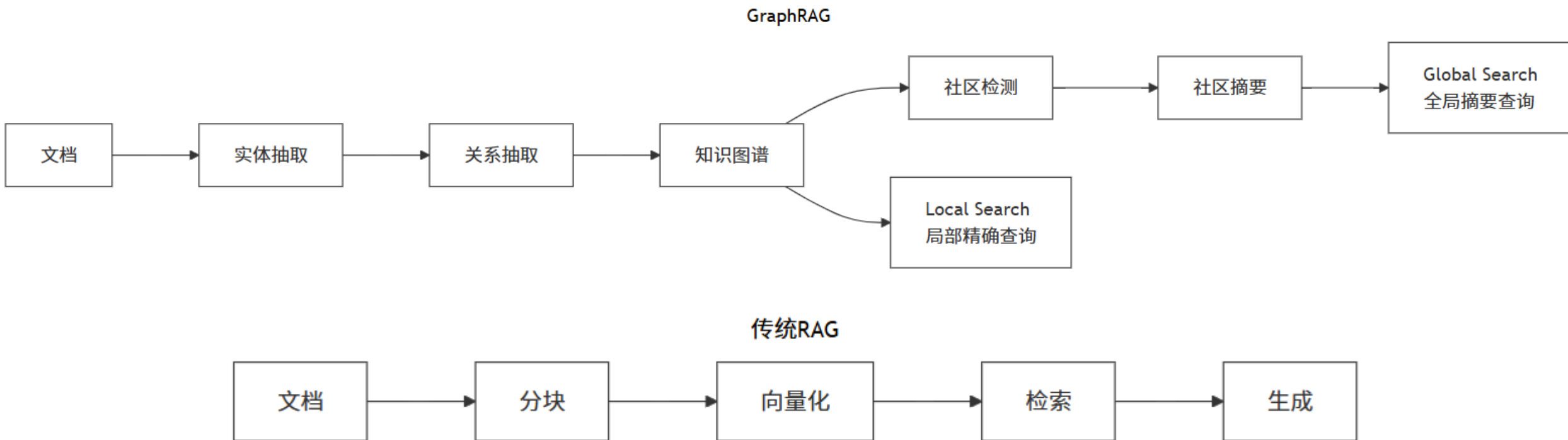
评估建议:

- 构建包含50-100个样本的评估集，覆盖各类问题
- 定期运行评估，监控系统质量变化
- 重点关注 Faithfulness，这是RAG的核心价值
- 结合定量指标和人工抽检

Q11: 什么是GraphRAG，与传统RAG的区别？

Q11: 什么是 GraphRAG，与传统RAG的区别？

GraphRAG 是微软提出的增强型RAG架构，通过构建知识图谱来增强检索和推理能力。



Q11: 什么是GraphRAG，与传统RAG的区别？

特性	传统RAG	GraphRAG
知识表示	文本块 + 向量	实体 + 关系 + 图结构
检索方式	语义相似度	图遍历 + 语义
全局问题	困难 (需要遍历所有文档)	擅长 (社区摘要)
多跳推理	弱	强 (沿关系推理)
构建成本	低	高 (需要实体抽取)
适用场景	直接问答	复杂推理、总结分析

Q11: 什么是GraphRAG，与传统RAG的区别？

Thinking: GraphRAG 中的核心概念都是什么？

- Entity (实体)

从文档中抽取的关键对象

例: 人名、地名、产品名、概念

- Relationship (关系)

实体之间的联系

例: "属于"、"制造"、"位于"

- Community (社区)

图中紧密相连的实体群组

通过社区检测算法发现

- Community Summary

每个社区的LLM生成摘要

用于回答全局性问题

Q11: 什么是GraphRAG，与传统RAG的区别？

Thinking: GraphRAG 中的两种查询模式都是什么？

- Local Search (局部搜索)

适合: "XXX公司的CEO是谁？" 这类精确问题

流程: Query -> 找到相关实体 -> 沿关系扩展 -> 收集上下文 -> 生成答案

- Global Search (全局搜索)

适合: "这篇文档的主要观点是什么？" 这类总结性问题

流程: Query -> 遍历社区摘要 -> Map-Reduce聚合 -> 生成综合答案

Q11: 什么是GraphRAG，与传统RAG的区别？

GraphRAG 使用建议:

- 构建成本高，适合高价值、复杂的知识库
- 对于简单FAQ，传统RAG已足够
- 可以与传统RAG结合: 简单问题用传统RAG，复杂问题用GraphRAG

面试高频问题汇总

Q13: RAG 和 Fine-tuning 怎么选?

选 RAG: 知识更新频繁、需要引用来源、数据量小、预算有限

选 Fine-tuning: 需要改变模型风格/格式、领域术语复杂、追求推理速度

组合使用: 先微调让模型更好地遵循检索结果, 再用RAG注入知识

面试高频问题汇总

Q14: 如何处理知识库中的矛盾信息?

为文档添加时间戳元数据, 优先使用最新的

为文档添加权威度标签, 优先使用官方来源

检索时同时返回多个来源, 让LLM综合判断

在Prompt中要求LLM指出信息冲突

面试高频问题汇总

Q15: RAG 系统的延迟优化有哪些方法?

- 向量检索: 使用ANN索引 (HNSW, IVF), 降低精确度换速度
- Embedding: 使用本地小模型, 或异步预计算
- Rerank: 减少候选数量, 或使用蒸馏小模型
- LLM: 使用流式输出, 选择更快的模型
- 缓存: 相似Query复用检索结果

面试高频问题汇总

Q16: 如何处理超长文档?

- 分层索引: 先检索摘要, 再检索详细段落
- 滑动窗口: 保留上下文的分块策略
- 长上下文模型: 使用支持128K+的模型 (如Qwen, Claude)
- 迭代检索: 先检索一部分, 根据LLM判断是否需要更多

面试高频问题汇总

Q17: 如何防止 LLM 幻觉?

- Prompt 明确指令: "仅基于提供的信息回答, 不确定时说不知道"
- 要求引用: 让LLM标注答案来源于哪个文档
- 降低 temperature: 减少随机性
- 答案验证: 用另一个LLM检查答案是否有上下文支撑
- Rerank 精选: 确保上下文高度相关

面试高频问题汇总

Q18: 多模态 RAG 怎么做?

- 图片: 使用多模态Embedding模型 (如 CLIP, 通义VL) 将图片向量化
- 表格: 转换为Markdown或JSON, 保持结构信息
- PDF: OCR提取文字 + 图表单独处理
- 视频: 抽帧 + 语音转文字, 分别建索引
- 统一使用多模态Embedding, 实现跨模态检索

面试高频问题汇总

Q19: 如何保证 RAG 系统的安全性?

- Prompt 注入防护: 过滤用户输入中的指令
- 权限控制: 根据用户角色过滤可检索的文档
- 敏感信息处理: 脱敏后入库, 或标记敏感级别
- 输出过滤: 检查生成内容是否包含敏感信息
- 审计日志: 记录所有查询和检索内容

打卡：RAG项目经历



准备1个自己做过的RAG项目，能讲述技术选型和踩坑经历

- RAG系统的架构
- 主流工具: LangChain, FAISS
- 关注最新技术: GraphRAG, Agentic RAG
- 理解评估指标，能说出如何衡量效果好坏
- 如何提升RAG的召回质量



Thank You
Using data to solve problems